

Backend

Verwendete Technologie

Python

Django

Django Rest Framework

Stripe

Postgres Datenbank

Gmail

Umgebung

Einstellungen (settings.py)

Umgebungsvariablen (.env)

Paketanforderungen (requirements.txt)

GitLab Runner

Produktionsumgebung

Docker Compose

Datenbank Migration

Apps

Users

Datenmodell

Endpunkte

Orders

Datenmodell

Endpunkte

Products

Datenmodell

Endpunkte

Payment

Endpunkte

Stripe Webhook

Email service

Konfiguration Google Account

Endpunkte

Verwendete Technologie

Python

Django

Da unsere Wahl auf Python fiel, haben wir uns für Django als Webframework entschieden, da es auf Python basiert und nahtlos damit zusammenarbeitet. Einer der Hauptgründe für die Wahl von Django ist seine Skalierbarkeit, die es ermöglicht, unsere Anwendung problemlos an steigende Anforderungen anzupassen. Außerdem bietet Django wichtige Sicherheitsfeatures wie integrierte Schutzmaßnahmen gegen gängige Webangriffe wie SQL-Injection, Cross-Site Scripting (XSS) und Cross-Site Request Forgery (CSRF), was die Sicherheit unserer Anwendung erheblich verbessert.

Django Rest Framework

Es handelt sich hierbei um eine Erweiterung von Django, die die Erstellung von APIs erleichtert. Grundlegend ermöglicht uns dies die Bereitstellung von RESTful Endpunkten, über die andere Systeme und unser Frontend.

Stripe

Als Zahlungsabwickler haben wir uns für Stripe entschieden, da der Anmeldeprozess sehr einfach ist und nur wenige Daten benötigt werden. Im Testmodus können bereits vollständige Zahlungen mit Testkreditkarten simuliert werden, was es uns ermöglicht, den gesamten Zahlungsprozess ohne Risiko zu testen. Stripe bietet zudem eine Vielzahl nützlicher Features, wie die Möglichkeit, direkt auf der Stripe-Seite Produkte anzulegen, auf die wir dann über die zugehörige Price IDs zugreifen können. Dies vereinfacht die Verwaltung von Produkten und gewährleistet eine sichere Zahlungsabwicklung.

Über das Stripe-Dashboard können wir alle wichtigen Leistungskennzahlen und Metriken einsehen, was uns wertvolle Einblicke in die Performance unseres Shops gibt. Mithilfe von Webhooks haben wir die Möglichkeit, verschiedene Events von Stripe abzugreifen und in Echtzeit in unserer Datenbank zu speichern. So können wir beispielsweise sicherstellen, dass die Zahlung erfolgreich durchgeführt wurde, indem wir entsprechende Daten sofort in unserer Datenbank vermerken und die Bestellprozesse automatisiert weiterführen. Stripe bietet uns dadurch nicht nur eine sichere und flexible Lösung, sondern auch wertvolle Tools zur Analyse und Optimierung unseres Geschäfts.

Postgres Datenbank

PostgreSQL bietet uns eine stabile und skalierbare Grundlage für die Speicherung und Verwaltung unserer Daten. Über das in Django integrierte ORM (Object-Relational Mapping) können wir auf einfache Weise mit PostgreSQL interagieren, ohne direkt SQL schreiben zu müssen.

Das ORM abstrahiert die Datenbanklogik und ermöglicht uns, effizient Daten zu Bestellungen, Zahlungen und Nutzern zu speichern und zu verwalten. Insbesondere durch die Integration von Stripe werden Zahlungsinformationen sicher in der Datenbank abgelegt.

Ebenso ist PostgreSQL dazu in der Lage, auch bei wachsenden Datenmengen eine gleichbleibende Leistung zu gewährleisten.

Gmail

Als E-Mail-Versand-Tool verwenden wir Gmail. Ein wesentlicher Grund dafür ist die einfache Einrichtung, der ausreichende Funktionsumfang für unsere Anforderungen sowie die Tatsache, dass es kostenlos zur Verfügung steht. Die Integration des Gmail-Kontos in unser Django-Backend erfolgt schnell und einfach durch Umgebungsvariablen unseres Gmail-Kontos und ermöglicht einen zuverlässigen Versand von E-Mails.

Gmail bietet zudem eine gute grafische Oberfläche, die es uns leicht macht, den E-Mail-Verkehr zu überwachen. Wir können jederzeit nachvollziehen, welche E-Mails versendet wurden, an wen sie gingen und wann der Versand dieser E-Mails stattfand. Ebenso können wir direkt in unserem Django-Backend die Emails konfigurieren, das heißt einen Empfänger, Betreff und den Nachrichteninhalt festlegen. Diese Informationen werden dann über unser Gmail-Konto an den jeweiligen Kunden weitergeleitet, wodurch ein reibungsloser und effizienter E-Mail-Versand sichergestellt wird.

Umgebung

Einstellungen ([settings.py](#))

Die `settings.py` Datei in Django steuert alle wesentlichen Konfigurationen unserer Anwendung. Hier legen wir wichtige Einstellungen wie die Datenbankverbindung, die installierten Apps und Middleware fest. Außerdem definieren wir sicherheitsrelevante Parameter, wie den CSRF-Schutz und die Allowed Hosts, um sicherzustellen, dass nur autorisierte Domains Zugriff auf unsere Anwendung haben. In Kombination mit den richtigen Caching- und Session-Optionen optimieren wir die Performance und Sicherheit unserer Anwendung.

Umgebungsvariablen (.env)

Um sensible Daten wie API-Schlüssel, Datenbank-Zugangsdaten oder Stripe-Keys sicher zu verwalten, verwenden wir Umgebungsvariablen, die in einer `.env` Datei gespeichert werden. Diese Datei enthält alle vertraulichen Konfigurationsparameter, die nicht direkt in den Code eingebunden werden sollen, um die Sicherheit unseres Backends zu erhöhen. Über Bibliotheken wie `python-decouple` greifen wir in der `settings.py` sicher auf diese Variablen zu.

Ein Beispiel unserer aktuellen .env sieht so aus:

```
SECRET_KEY='django-insecure-xxxxxx'
```

```
POSTGRES_DB=shopcore-db
```

```
POSTGRES_USER=xxxxxx
```

```
POSTGRES_PASSWORD='xxxxxxx'
```

```
STRIPE_SECRET_KEY='sk_test_xxxxxxxxxxx'
```

```
STRIPE_PUBLISHABLE_KEY='pk_test_xxxxxx'
```

```
STRIPE_SITE_URL='http://localhost:3000/'
```

```
EMAIL_HOST='smtp.gmail.com'
```

```
EMAIL_PORT=587
```

```
EMAIL_USE_TLS=True
```

```
EMAIL_HOST_PASSWORD='xxxxx'
```

```
DEFAULT_FROM_EMAIL='shopcore.info@gmail.com'
```

Einige Variablen und deren Werte unterscheiden sich hierbei natürlich zwischen der lokalen Entwicklungsumgebung und der Produktivumgebung. Eine etwas detaillierte Beschreibung hierzu folgt unter dem Punkt Produktivumgebung.

Paketanforderungen (requirements.txt)

Die `requirements.txt` enthält alle Python-Pakete, die unsere Anwendung benötigt. In der lokalen, sowie der Produktionsumgebung, welche beide mit Docker betrieben werden, sorgt die `requirements.txt` dafür, dass alle Abhängigkeiten konsistent installiert werden. In der Docker-Datei (`Dockerfile`) wird während des Build-Prozesses `RUN pip install -r requirements.txt` verwendet, um die Pakete aus der `requirements.txt` zu installieren. Dies gewährleistet, dass die Python-Abhängigkeiten stets identisch sind, unabhängig von der Umgebung, in der der Container ausgeführt wird. Der Vorteil hierbei besteht darin, dass nun keine Abhängigkeiten/Pakete mehr manuell installiert/geupdated werden müssen.

GitLab Runner

Der GitLab Runner automatisiert unsere CI/CD-Pipeline (besonders sinnvoll für unsere Produktivumgebung), um den gesamten Build und Deploymet-Prozess zu vereinfachen. Jedes Mal, wenn Änderungen am Code vorgenommen und in das GitLab-Repository gepusht werden, startet der Runner die Prozesse (`main` branch). Dabei werden die Docker-Container, basierend auf der `docker-compose.yml` Datei gestartet. Der GitLab Runner sorgt dafür, dass die neueste Version

des Codes in der Produktionsumgebung deployed wird, einschließlich der Aktualisierung von Abhängigkeiten aus der `requirements.txt`.

Dadurch ist nun jeder Entwickler in der Lage, die neusten Änderungen auf die Produktivumgebung zu bringen und diese live zu testen.

Produktionsumgebung

Unsere Produktionsumgebung läuft in einem isolierten Docker-Container auf einem Linux-basierten Server. Dieser Container enthält die vollständige Backend-Anwendung sowie alle Abhängigkeiten, die in der `requirements.txt` definiert sind. Um diese Produktionsumgebung sicher und stabil zu betreiben, haben wir mehrere Schritte durchgeführt:

- **Firewall-Konfiguration (UFW und CrowdSec):**

- Eine der ersten Maßnahmen war das Einrichten einer Firewall (UFW – Uncomplicated Firewall), um nur die notwendigen Ports freizugeben, wie beispielsweise HTTP (80), HTTPS (443) und SSH (22).

Zusätzlich haben wir CrowdSec integriert, CrowdSec erkennt verdächtige Aktivitäten in Echtzeit und blockiert potenziell gefährliche IP-Adressen. Durch seinen kollaborativen Ansatz, der Bedrohungsinformationen aus der ganzen Welt sammelt, erhöht CrowdSec den Schutz vor globalen Angriffen, während UFW lokale Richtlinien für den Netzwerkzugang festlegt.

Generell liegt dieser Server noch hinter einem Gateway und ist durch zusätzliche Netzwerk relevanten Sicherheitstechniken geschützt.

- **SSH-Zugriffe:**

- Der SSH-Zugriff auf den Server wurde abgesichert, indem wir Passwort-Authentifizierungen deaktiviert und auf Schlüssel-basierte Authentifizierung gesetzt haben. Dies minimiert beispielsweise das Risiko von Brute-Force-Angriffen. Das verwenden von SSH ist in unserem fall notwendig um zugriff auf die Produktivumgebung zu haben.

- **Isolieren der Umgebungen:**

- Grundsätzlich isoliert Docker die Container, wir haben aber die verschiedenen Services (wie Backend, Datenbank und Nginx) nochmals in separaten Containern laufen. Diese Isolation sorgt dafür, dass Probleme in einem Container nicht auf andere übergreifen und macht die gesamte Umgebung stabiler und wartbarer.

- **SSL-Zertifikat und Nginx Proxy Manager:**

- Um die Sicherheit der Kommunikation zwischen dem Server und den Nutzern zu gewährleisten, haben wir ein SSL-Zertifikat mithilfe von dem Nginx Proxy Manager

aufgesetzt. Dieser ermöglicht es uns SSL-Verschlüsselung über Let's Encrypt bereitzustellen und unsere Anwendung sicher über HTTPS auszurollen.

- **Domain (IONOS):**

- Für den externen Zugriff auf unsere Anwendung haben wir eine Domain bei IONOS. Die Domain ist mit unserem Server verbunden, sodass unsere Anwendung unter einer benutzerfreundlichen URL erreichbar ist. Die Kombination aus Domain und SSL-Zertifikat stellt sicher, dass die Benutzer über eine gesicherte Verbindung auf unsere Anwendung zugreifen können.

Da wir lokal entwickeln und gleichzeitig eine Produktivumgebung haben, haben wir eine zusätzliche umgebungsvariable `IS_PRODUCTION` definiert um verschiedene Einstellungen der `settings.py` in der jeweiligen Umgebung zu differenzieren. Ein Beispiel hierfür wäre der folgende Teil:

```
IS_PRODUCTION = config('IS_PRODUCTION', default=False, cast=bool)
```

```
if IS_PRODUCTION:
```

```
    SESSION_COOKIE_SECURE = True
```

```
    CSRF_COOKIE_SECURE = True
```

```
    DEBUG = False
```

```
    SESSION_COOKIE_SAMESITE = 'None'
```

```
    CSRF_COOKIE_SAMESITE = 'None'
```

```
    SESSION_COOKIE_DOMAIN = '.lensim.de'
```

```
    CSRF_COOKIE_DOMAIN = '.lensim.de'
```

```
else:
```

```
    SESSION_COOKIE_SECURE = False
```

```
    CSRF_COOKIE_SECURE = False
```

```
    DEBUG = True
```

```
    SESSION_COOKIE_SAMESITE = 'Lax'
```

```
    CSRF_COOKIE_SAMESITE = 'Lax'
```

```
    SESSION_COOKIE_DOMAIN = 'localhost'
```

```
    CSRF_COOKIE_DOMAIN = 'localhost'
```

Um nun unsere Applikationen im Internet zu erreichen, gibt es zwei Adressen:

Backend □ <https://shop-api.lensim.de/api/v1/>[+endpoint URL]

Frontend □ <https://shop-frontend.lensim.de/>

Hierbei ist das **Backend** nur über unser **Frontend** erreichbar und nicht öffentlich zugänglich, somit akzeptiert es nur Anfragen, welche vom Frontend geschickt werden.

Docker Compose

Docker ermöglicht es uns, eine stabile und einheitliche Umgebung bereitzustellen, die auf allen Systemen gleich läuft - unabhängig von den installierten Systembibliotheken.

Datenbank Migration

In Django muss bei jeder Änderung an den Modellen eine Migration durchgeführt werden, damit die Datenbankstruktur mit den Modellen synchron bleibt und die Daten konsistent sind. Dies betrifft Änderungen wie das Hinzufügen, Entfernen oder Ändern von Feldern, sowie die Anpassung von Datentypen in den Modellen. Migrationen sind wichtig, damit Änderungen im Code auch in die Datenbank reflektiert werden.

Da wir in einem Docker Compose Setup arbeiten, sind die Befehle für die Migration etwas anders als bei einer Standardumgebung.

Der Befehl, um eine neue Migration zu erstellen, lautet:

```
docker compose exec web python manage.py makemigrations
```

Dieser Befehl analysiert die vorgenommenen Änderungen in den Modellen und erstellt die entsprechenden Migrationsdateien, die dann die Datenbankstruktur anpassen. Sobald die Migration erstellt wurde, muss sie noch auf die Datenbank angewandt werden. Dies geschieht mit dem folgenden Befehl:

```
docker compose exec web python manage.py migrate
```

Wenn nur Änderungen an einer spezifischen App, wie z. B. dem Product-Modell vorgenommen wurden, kann auch gezielt eine Migration für diese App erstellt werden, das geht durch die Mitgabe des Namens im Befehl:

```
docker compose exec web python manage.py makemigrations products
```

Apps

In den folgenden aufgelisteten Endpunkten ist wichtig zu erwähnen, dass jedes Element, sei es ein User, eine Order,... eine eindeutige id hat. Diese id wird von django direkt vergeben und ist deshalb nicht im Datenmodell zu sehen.

Users

Datenmodell

```
class CustomUser(AbstractBaseUser, PermissionsMixin):
    email = models.EmailField(unique=True, blank=False)
    first_name = models.CharField(max_length=30, blank=False)
    last_name = models.CharField(max_length=30, blank=False)
    date_of_birth = models.DateField(null=True, blank=True)
    phone_number = models.CharField(max_length=15, blank=True)
    street_and_number = models.CharField(max_length=50, blank=True)
    postal_code = models.CharField(max_length=7, blank=True)
    city = models.CharField(max_length=50, blank=True)
    additional_address_info = models.CharField(max_length=255, blank=True)
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)

    objects = CustomUserManager()

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['first_name', 'last_name', 'phone_number']
```

Pflichtfelder in diesem Kontext sind:

`email` (standardmäßig von django), `first_name`, `last_name`, `phone_number`

Endpunkte

<https://shop-api.lensim.de/api/v1/>

GET all users □ `auth/`

- Dieser Endpunkt gibt alle User zurück, auf diesen Endpunkt können nur Nutzer mit mindestens `is_staff = true` zugreifen.

GET user info □ `auth/info/`

- Dieser Endpunkt gibt die Nutzerinformation des aktuellen Nutzer zurück. Vorausgesetzt der Nutzer ist authentifiziert.

POST logout □ `auth/logout/`

- Nutzer wird ausgeloggt, vorausgesetzt der Nutzer ist authentifiziert.

POST login □ `auth/login/`

- Nutzer wird eingeloggt, vorausgesetzt der Nutzer ist ausgeloggt.

POST register □ `auth/register/`

- Neuer Nutzer wird erstellt, keine Voraussetzungen.

GET a single user □ `auth/{userId}/`

- Gibt den Nutzer mit der gegebenen id zurück, kann nur von einem authentifizierten Nutzer mit seiner eigenen id ausgeführt werden. Ebenfalls mit `is_staff = true` für alle id's.

PUT a single user □ `auth/{userId}/`

- Updated die Nutzerinformation, kann nur von einem authentifizierten Nutzer mit seiner eigenen id ausgeführt werden. Ebenfalls mit `is_staff = true` für alle id's.

PATCH a single user `auth/{userId}/`

- Updated die Nutzerinformation, kann nur von einem authentifizierten Nutzer mit seiner eigenen id ausgeführt werden. Ebenfalls mit `is_staff = true` für alle id's.

DELETE a single user `auth/{userId}/`

- Löscht den Nutzer, kann nur mit `is_staff = true` für alle id's erledigt werden.

Orders

Datenmodell

```
class Order(models.Model):
    STATUS_CHOICES = (
        ('new', 'New order'),
        ('in_progress', 'In progress'),
        ('in_shipment', 'In Shipment'),
        ('delivered', 'Delivered'),
        ('cancelled', 'Cancelled'),
    )
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='new')
    alternative_address = models.JSONField(default=dict, null=True, blank=True)
    city = models.CharField(max_length=100)
    country = models.CharField(max_length=100)
    differentShippingAddress = models.BooleanField(default=False)
    email = models.EmailField()
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)
    phone_number = models.CharField(max_length=20)
    postal_code = models.CharField(max_length=20)
    payedStatus = models.CharField(max_length=100, blank=True)
    payed_at = models.DateTimeField(auto_now=True)
    paymentMethod = models.CharField(max_length=100, blank=True)
    salutation = models.CharField(max_length=10) # Mr., Ms., Mx., etc.
    selectedProducts = models.JSONField(default=dict)
    street_and_number = models.CharField(max_length=255)
    termsAccepted = models.BooleanField(default=False)
    totalPrice = models.DecimalField(max_digits=20, decimal_places=2)
    orderPlacedByUserWithId = models.CharField(max_length=255, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

Die Felder `id`, `created_at` und `updated_at` sind schreibgeschützte Felder, d. h., sie können nicht verändert werden. Auch das Feld `orderPlacedByUserWithId` wird beim Erstellen einer Bestellung vom

Backend automatisch gesetzt. Dies haben wir aus Sicherheitsgründen so implementiert, damit die Bestellung stets dem korrekten Benutzer zugeordnet wird.

Endpunkte

<https://shop-api.lensim.de/api/v1/>

GET all Orders □ `orders/`

- Dieser Endpunkt gibt alle Orders zurück, auf diesen Endpunkt können nur Nutzer mit mindestens `is_staff = true` zugreifen.

GET single Order □ `orders/{orderId}/`

- Dieser Endpunkt gibt eine einzelne Order zurück, auf diesen Endpunkt können nur Nutzer mit mindestens `is_staff = true` zugreifen.

GET orders from User □ `orders/user/{userID}/`

- Dieser Endpunkt gibt alle Orders eines bestimmten Users zurück, auf diesen Endpunkt können nur Nutzer mit mindestens `is_staff = true` zugreifen oder der authentifizierte User dem diese Orders gehören, also wenn die userID des authentifizierten Users mit der in der Order übereinstimmt.

POST new Order □ `orders/`

- Dieser Endpunkt erstellt eine neue Order, neue Orders können von absolut allen Nutzern der Website erstellt werden, der Nutzer muss dafür auch nicht authentifziert sein.

PUT update Order □ `orders/{orderId}/`

- Dieser Endpunkt aktualisiert die Informationen der entsprechenden Order, auf diesen Endpunkt können nur Nutzer mit mindestens `is_staff = true` zugreifen.

DELETE delete Order □ `orders/{orderId}/`

- Dieser Endpunkt löscht die entsprechende Order, auf diesen Endpunkt können nur Nutzer mit mindestens `is_staff = true` zugreifen.

Products

Datenmodell

```
class Product(models.Model):
    SIZE_CHOICES = (
        ('S', 'S'),
        ('M', 'M'),
        ('L', 'L'),
        ('XL', 'XL'),
    )

    name = models.CharField(max_length=100)
    description = models.TextField(null=True)
```

```

price = models.DecimalField(max_digits=10, decimal_places=2)
stripePriceId = models.TextField(max_length=200, null=True)
stock = models.JSONField(default=dict) # Example: {"XS": 10, "S": 15, "M": 5}
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
productBlenderModel = models.FileField(upload_to='blender_models/', blank=True, null=True)
productPicture = models.ImageField(upload_to='product_pictures/', blank=True, null=True)
colors = models.JSONField(default=list) # Example: ["Red", "Blue", "Green"]
category = models.JSONField(default=list)
discount = models.BooleanField(default=False)
discountInPercent = models.PositiveIntegerField(default=0)

```

Endpunkte

<https://shop-api.lensim.de/api/v1/>

GET all products □ `products/`

- Es können alle Produkte ohne Authentifizierung abgerufen werden.

POST product □ `products/`

- Ein neues Produkt kann nur mit mindestens der Berechtigung `is_staff = true` erstellt werden.

GET a single product □ `products/{productID}`

- Ein einzelnes Produkt kann ohne Authentifizierung abgerufen werden.

PUT a single product □ `products/{productID}`

- Ein Produkt kann nur mit mindestens der Berechtigung `is_staff = true` aktualisiert werden.

PATCH a single product □ `products/{productID}`

- Ein Produkt kann nur mit mindestens der Berechtigung `is_staff = true` aktualisiert werden.

DELETE a single product □ `products/{productID}`

- Ein Produkt kann nur mit mindestens der Berechtigung `is_staff = true` gelöscht werden.

Payment

Endpunkte

<https://shop-api.lensim.de/api/v1/>

POST create Stripe Checkout □ `payments/create-checkout-session/`

- Eine Stripe Checkout Session kann von jedem User, auch ohne Authentifizierung erstellt werden.

POST retrieve Changes from Stripe □ `payments/webhook/`

- Die Änderungen bei Stripe werden direkt von Stripe an unseren Endpunkt gesendet.

Stripe Webhook

Wir nutzen einen Stripe Webhook, um den Status unserer Bestellungen nach Abschluss des Checkouts automatisch zu aktualisieren. Wenn das Event `checkout.session.completed` von Stripe gesendet wird, aktualisieren wir den `payedStatus` der entsprechenden Bestellung auf `true` und setzen das Feld `payed_at` auf den Zeitpunkt der erfolgreichen Zahlung. Sollte der Nutzer den Stripe-Checkout verlassen, ohne die Transaktion abzuschließen, sendet Stripe nach 24 Stunden das Event `checkout.session.expired`. In diesem Fall setzen wir den Bestellstatus auf `cancelled`. Mit diesem automatisierten Prozess stellen wir sicher, dass der Zahlungsstatus, als auch der allgemeine Status der Bestellungen immer aktuell und zuverlässig ist.

Email service

Konfiguration Google Account

Um das senden von Emails umzusetzen, wurde ein Gmail Account angelegt (shopcore.info@gmail.com), unter welchem ein neues App-Passwort für den django service festgelegt wurde. Dies erlaubt es uns Emails per API zu versenden.

Endpunkte

<https://shop-api.lensim.de/api/v1/>

POST a new Mail □ `send-email/`

- Versendet eine Email an einen Nutzer (jeweilige Email) mit Betreff und Email Inhalt. Kann nur mit mindestens `is_staff = true` ausgeführt werden. Hiervon ausgeschlossen, sind automatisierte Emails, welche bspw. beim Kaufprozess abgesendet werden. Die Berechtigungen beziehen sich also lediglich auf diesen Endpunkt.